

NeuroForge — Team Setup Guide (Docker + CI/CD)

This covers everything needed to get the project running locally via Docker, and to understand how the GitHub Actions pipeline (Trigger/Rollback) works. Follow these in order — later steps depend on earlier ones.

1. Pull the latest code

Make sure you have the latest `main`, including: - `docker-compose.yml` (backend service now has `GITHUB_PAT`, `GITHUB_OWNER`, `GITHUB_REPO` env vars) - `.github/workflows/ci-cd.yml` (Jenkins removed, GitHub Actions does build+test+deploy+rollback) - `Backend/src/main/java/.../config/RestTemplateConfig.java` (new — registers the `RestTemplate` bean used to call GitHub's API) - `Backend/src/main/resources/application.properties` (new `github.*` properties)

```
git pull origin main
```

2. Create your local `.env` file

This file is **git-ignored** — each teammate creates their own, it is never committed. Create `.env` in the project root (same folder as `docker-compose.yml`):

```
# Database
DB_PASSWORD=choose_any_password_you_like

# GitHub integration (see step 3 for how to get GITHUB_PAT)
GITHUB_PAT=ghp_XXXXXXXXXXXXXXXXXXXXXXXXXXXXX
GITHUB_OWNER=<repo-owner-username>
GITHUB_REPO=NeuroForge
```

Double-check `.env` is listed in `.gitignore`. If it isn't, add it now — committing `GITHUB_PAT` would leak a live token into git history.

3. Get your own GitHub Personal Access Token (PAT)

The backend uses this token to call GitHub's API when someone clicks **Trigger Build** or **Rollback** on the dashboard. Each teammate running the backend locally needs their **own** token (don't share one person's token around).

1. Go to **github.com/settings/tokens** → **Fine-grained tokens** → **Generate new token**
2. Name it something like `neuroforge-local`
3. Set an expiration (90 days is fine — you'll regenerate later)
4. **Repository access** → **Only select repositories** → choose `NeuroForge`
5. **Permissions** → **Repository permissions** → **Actions** → set to **Read and write**
6. Generate → copy the token immediately (GitHub only shows it once)
7. Paste it into your `.env` as `GITHUB_PAT` (step 2 above)

You need **write access to the repo** to create this token with Actions permissions — ping whoever owns the repo if you don't have that yet.

4. Run the project locally

```
docker compose up --build
```

This starts, in order (each waits on the previous being healthy): `postgres` → `keycloak` → `kafka` → `backend` → `frontend`

First boot is slow — Keycloak's health check alone can take a few minutes. Once up: - Frontend: **http://localhost:5173** - Backend API: **http://localhost:9000** - Keycloak admin: **http://localhost:8080** (login: `admin` / `admin`)

If `backend` fails to start immediately, it's almost always a missing/incomplete `.env` — recheck step 2 first before debugging code.

5. Understanding the CI/CD side (GitHub Actions)

Jenkins is fully gone. Everything now runs as a single GitHub Actions workflow (`.github/workflows/ci-cd.yml`), triggered two ways:

- **Automatically** — every push to `main`
- **Manually** — clicking **Trigger Build** or **Rollback** on the dashboard (this calls the backend, which calls GitHub's `workflow_dispatch` API using your `GITHUB_PAT`)

What the workflow does on a normal run: 1. Checkout → build & test with Maven (against a throwaway Postgres container) 2. Build a Docker image, tag it with the commit SHA 3. Push it to

GitHub Container Registry (`ghcr.io/<repo-owner>/neuroforge:<sha>`) 4. Run the container, health-check it 5. Report results back to the dashboard's webhook endpoint

On a **rollback** run, it skips build/test/push entirely and just pulls a previously-pushed image tag and redeploys it — this only works because every normal run pushes its image to GHCR first (that's what makes an old version available to roll back to).

Repo-level things already set up (nothing you need to do here, just FYI)

- **Repo secret** `DB_PASSWORD` — used by the workflow's Postgres test service
- **Repo secret** `CONTROLLER_URL` — points at wherever the dashboard's webhook endpoint is reachable (currently an ngrok tunnel, since nothing is deployed yet — see note below)
- **Settings → Actions → General → Workflow permissions** is set to **Read and write** (required for the workflow to push images to GHCR)
- The workflow itself has `permissions: packages: write` declared at the top

You should **not** need to touch any of these — they're repo-wide, not per-person.

6. Known temporary limitation

Nothing is deployed yet, so `CONTROLLER_URL` currently points at whichever teammate's local machine is running the backend + an active `ngrok http 9000` tunnel. That means webhook delivery (build results showing up on the dashboard) only works while that person's tunnel is running. This goes away once we deploy the backend somewhere permanent — until then, coordinate who's "hosting" the controller before testing Trigger/Rollback end-to-end.

Quick troubleshooting checklist

Symptom	Likely cause
Backend won't start	<code>.env</code> missing or incomplete (step 2)
Trigger/Rollback button fails silently	Your <code>GITHUB_PAT</code> is missing, expired, or lacks Actions write permission
Dashboard never updates after a build	<code>CONTROLLER_URL</code> tunnel isn't running / has rotated (ngrok free URLs change on restart)
<code>docker compose up</code> hangs on Keycloak	Normal — its health check window is long (up to 10 min) on first boot